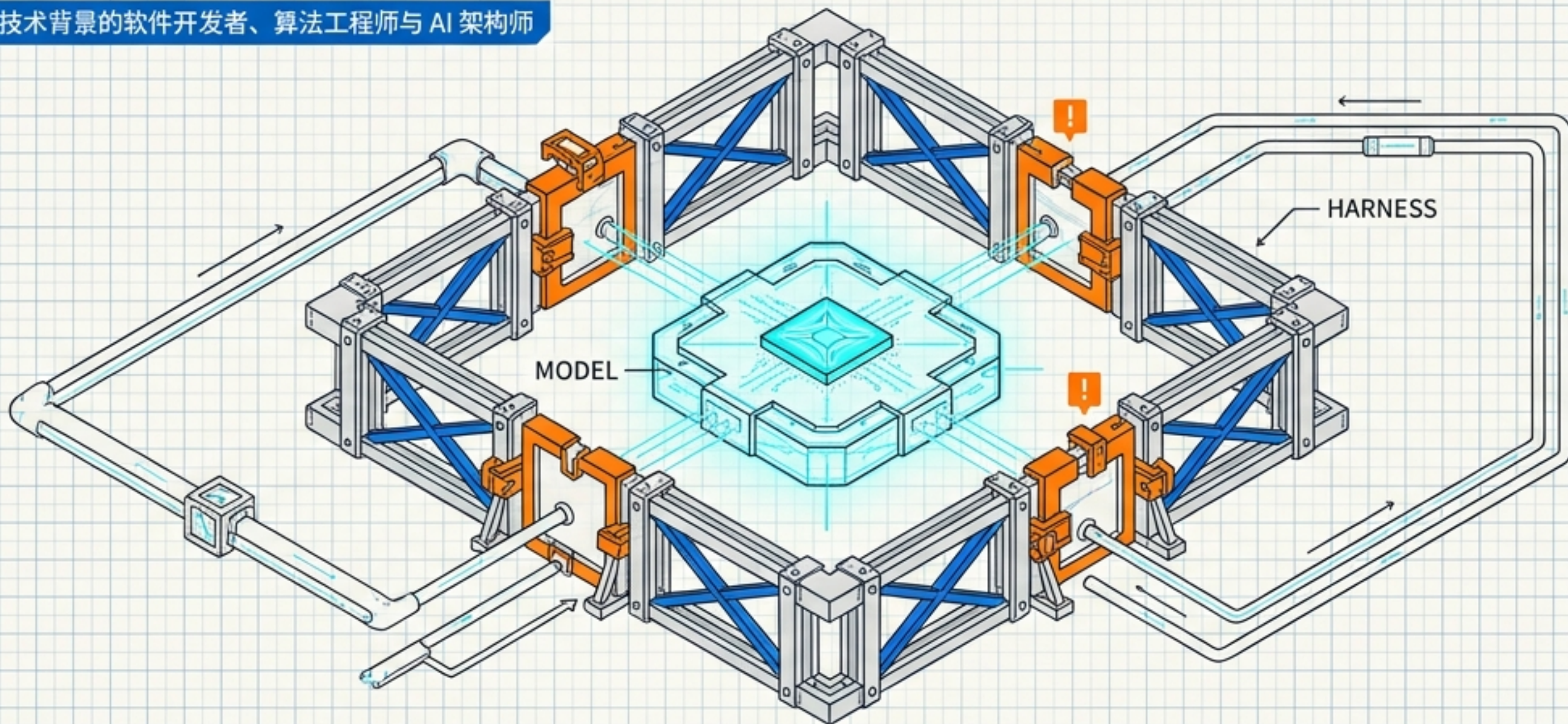


从“缸中之脑”到受控数字工人

Agent 平台架构、Skill 工程与生产级 Vibe Coding 蓝图

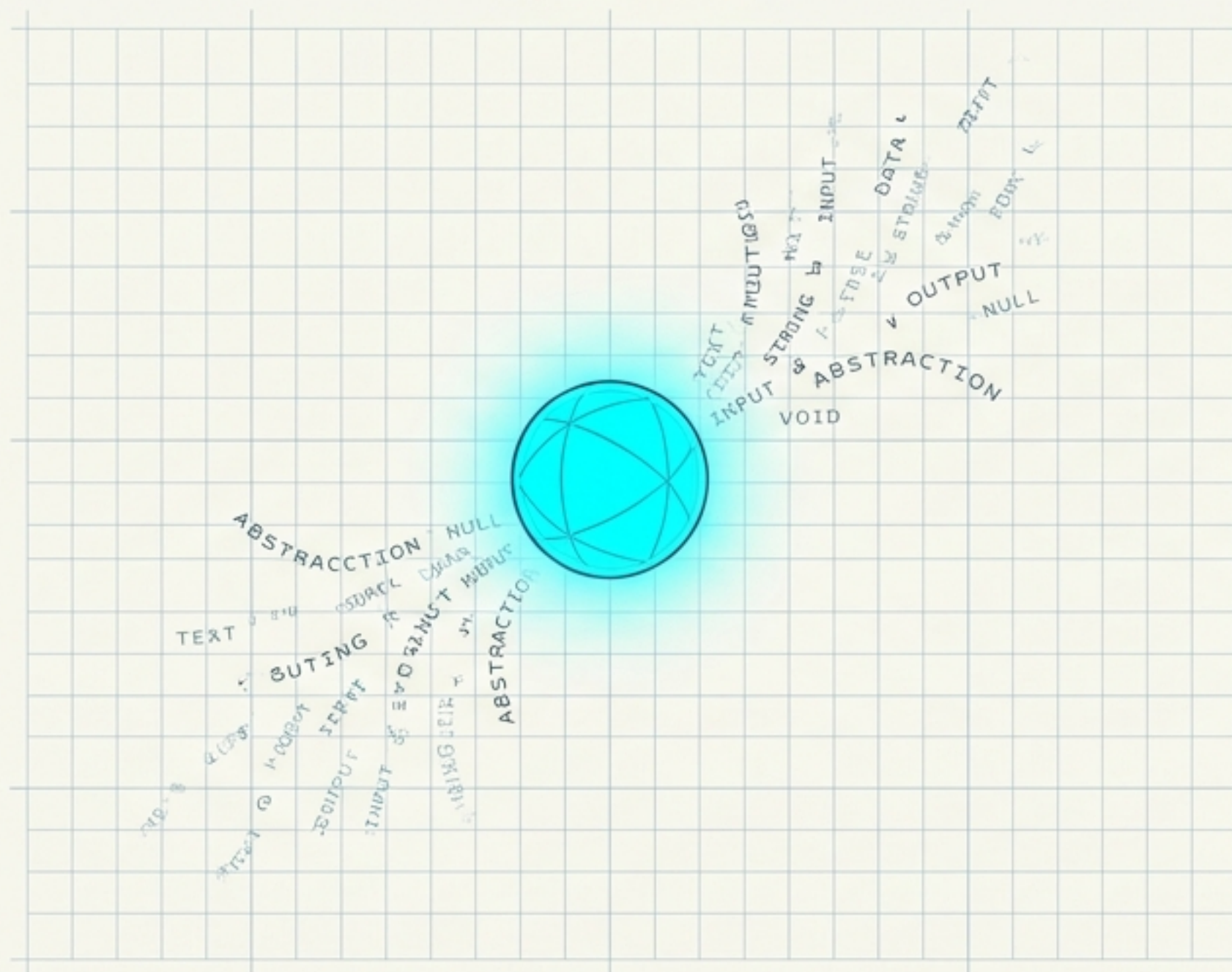
目标受众：具有技术背景的软件开发者、算法工程师与 AI 架构师



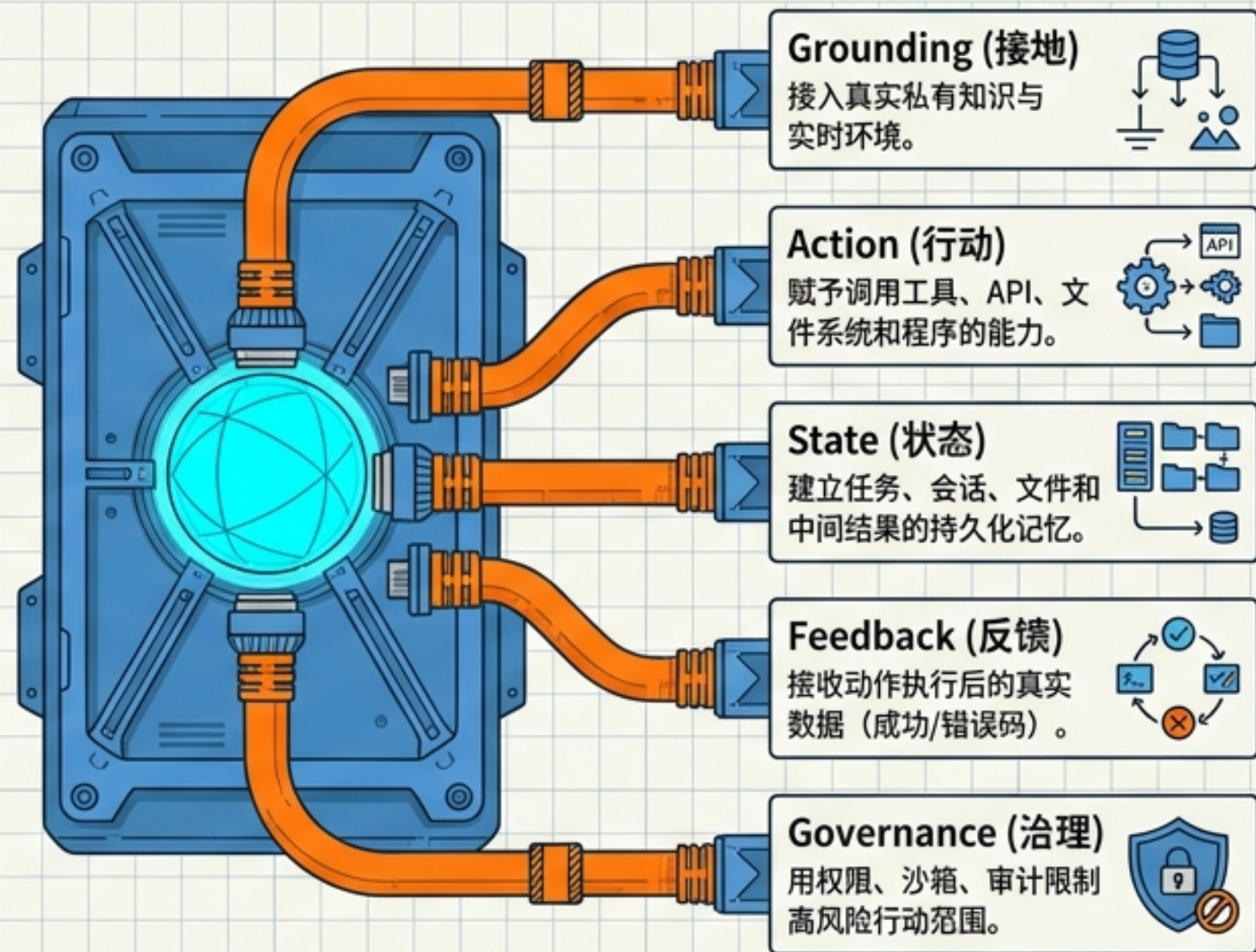
模型能力决定 Agent 的认知上限；运行脚手架（Harness）、权限与验证闭环，决定它能否稳定进入生产。

纯语言模型的困境：为什么需要外部脚手架？

缸中之脑



受控数字工人



从 Chatbot 到 Agent 不是名词的升级，而是模型与真实世界之间“接口”的补齐。

架构演进：从聊天框到受控工作区

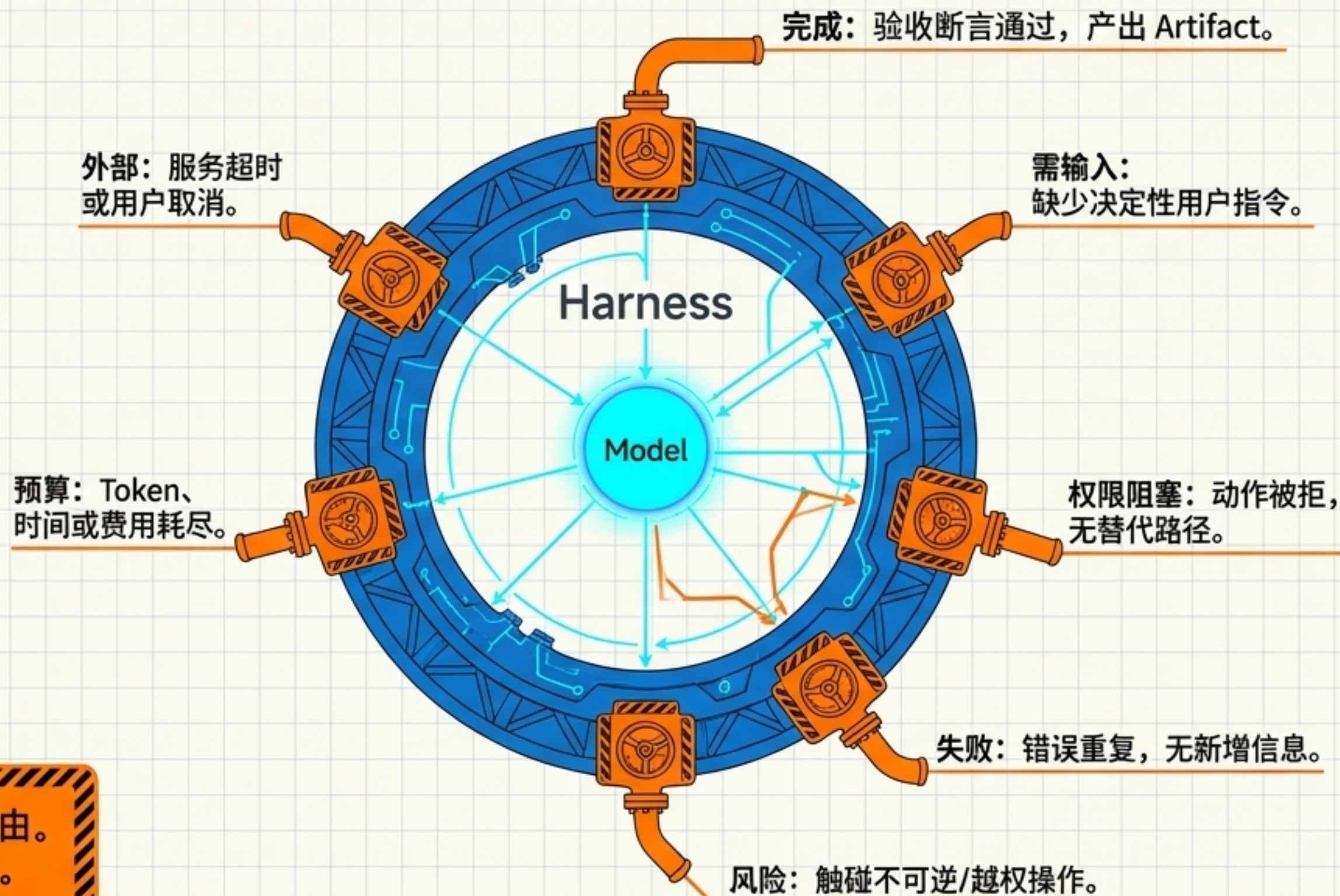
阶段	控制者	补齐的关键能力	典型局限
Chatbot	用户	自然语言交互	只能“说”，无执行力
RAG + Chat	检索程序	外部与私有知识	仅解决知识缺口，不执行任务
Workflow + Chat	开发者预定义图	确定性编排与系统集成	长尾需求导致异常分支无限增加
Agent + Workspace	模型在 Harness 边界内	动态规划、工具选择、状态延续	验证机制复杂

洞察：演进不是消除确定性。合理的系统分工是：把不确定性留给模型，把确定性沉淀到环境（Harness）。

核心工程公式：Agent = Model + Harness

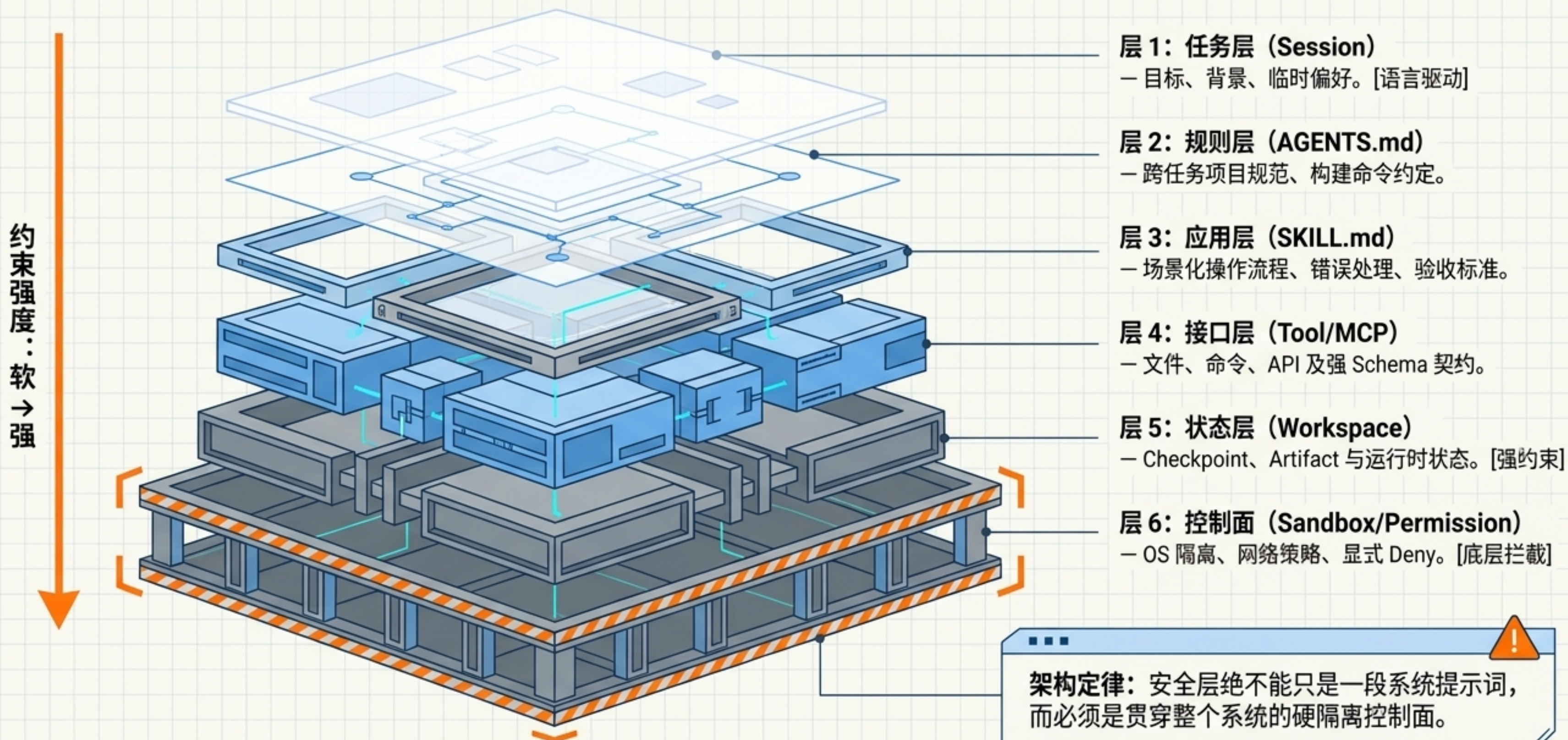
Harness 核心职责

- 组织上下文与暴露结构化工具
- 拦截越权与管理执行状态
- 处理重试、超时、压缩与恢复



警告：“还能继续想”不是继续循环的理由。
模型越自主，Harness 的兜底责任越重。

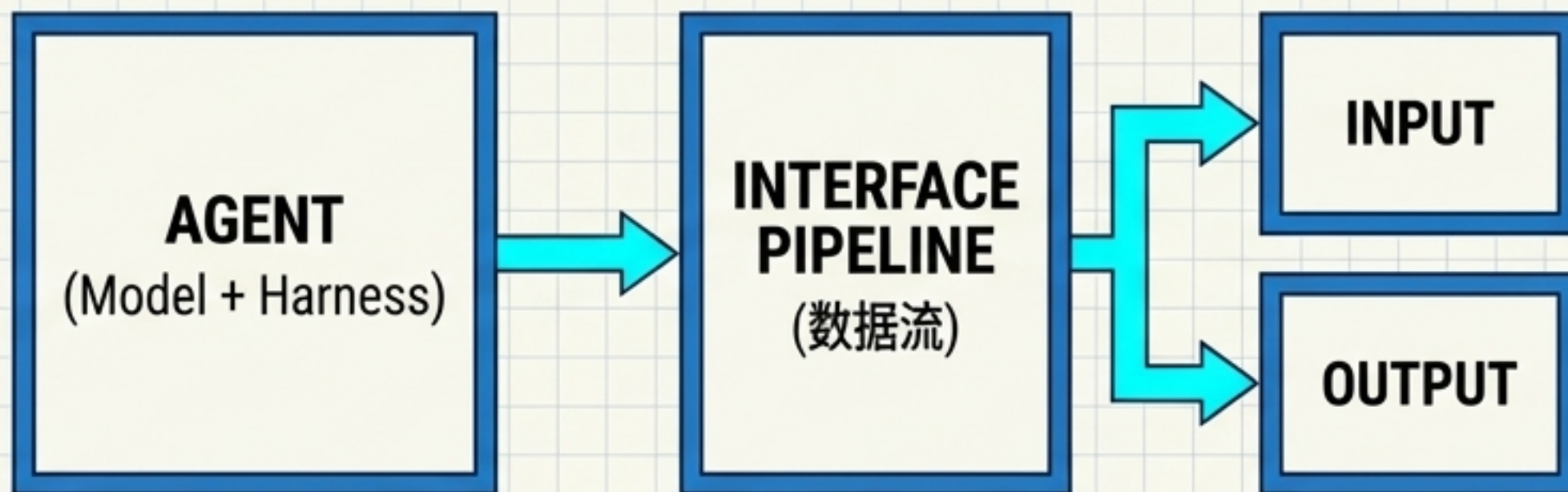
生产级 Agent 平台：六层等距架构切片



动作契约：Tool 与 Script 的工程标准

Tool 设计 5 原则

1. 窄接口
2. 强 Schema
3. 可诊断错误
4. 安全重试（幂等）
5. 结果可验证

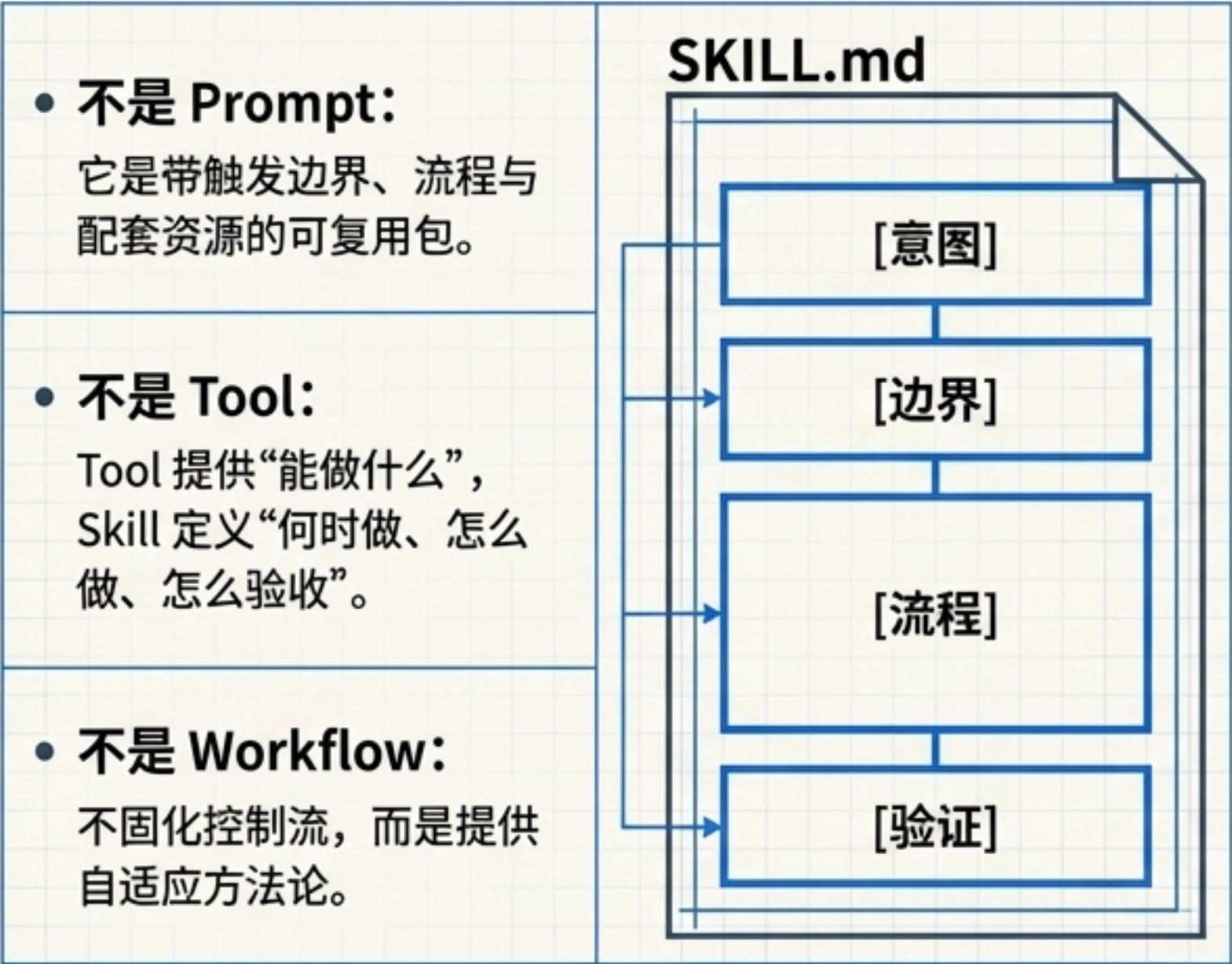


为 Agent 设计的 I/O 规范

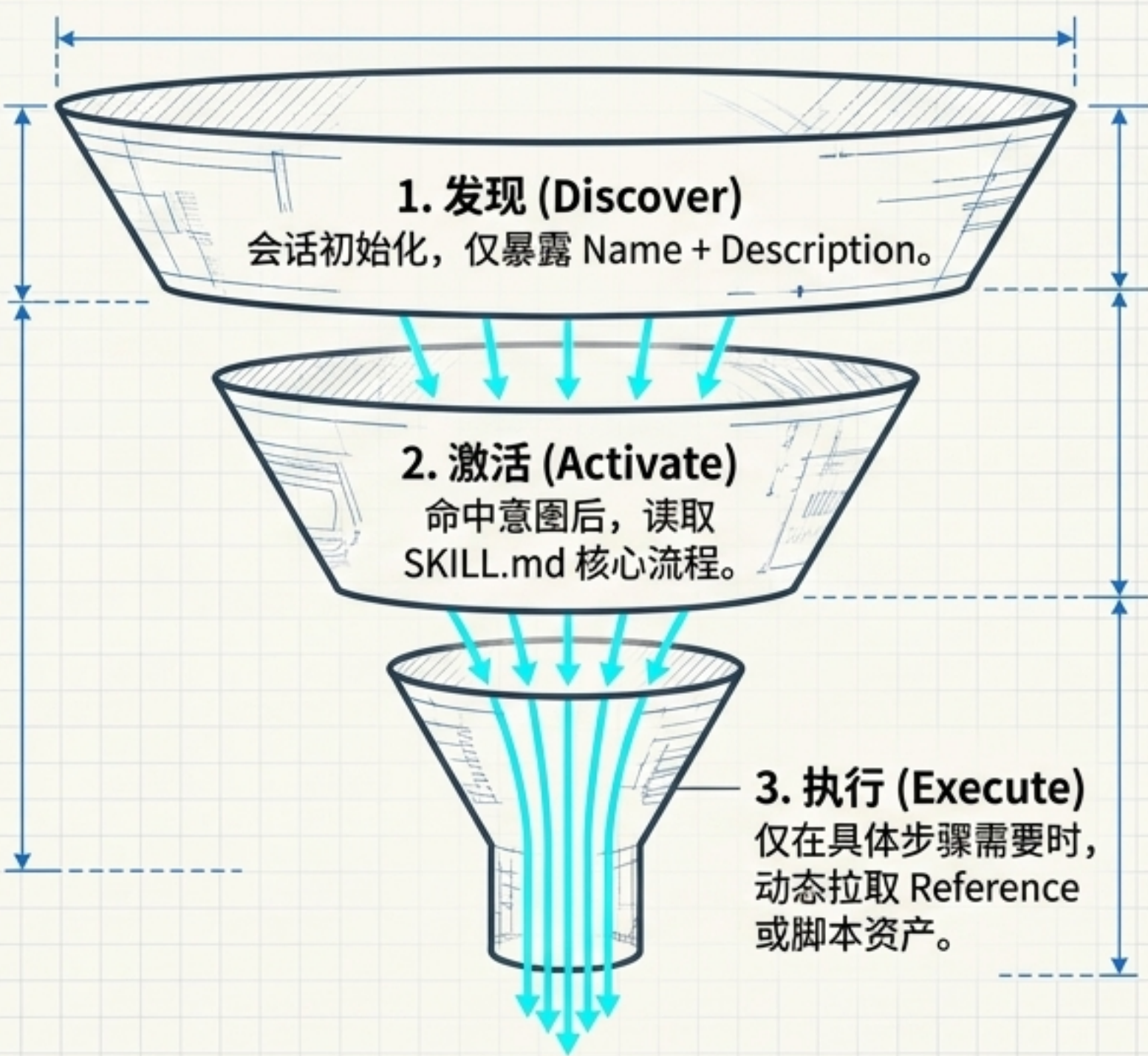
stdin/参数	拒绝等待 TTY 交互；复杂对象采用单一 JSON。
stdout（机器结果）	仅输出一个合法 JSON 结构体，严禁混入进度文字。
stderr（诊断日志）	承载所有警告、进度条与人类可读错误日志。
exit code（失败分类）	必须为鉴权、参数、业务冲突设置稳定错误码。
output file	大表格/图像写入文件，stdout 仅返回 Manifest。

技能解剖：什么是可按需加载的 Skill？

Skill 不是什么



渐进式披露漏斗（按需加载）



状态与记忆：从临时对话到长期资产

Memory Timeline



当前输入与短工具结果
(支持下一步决策)。

任务处于
planning/working/
blocked, 已确认目标。

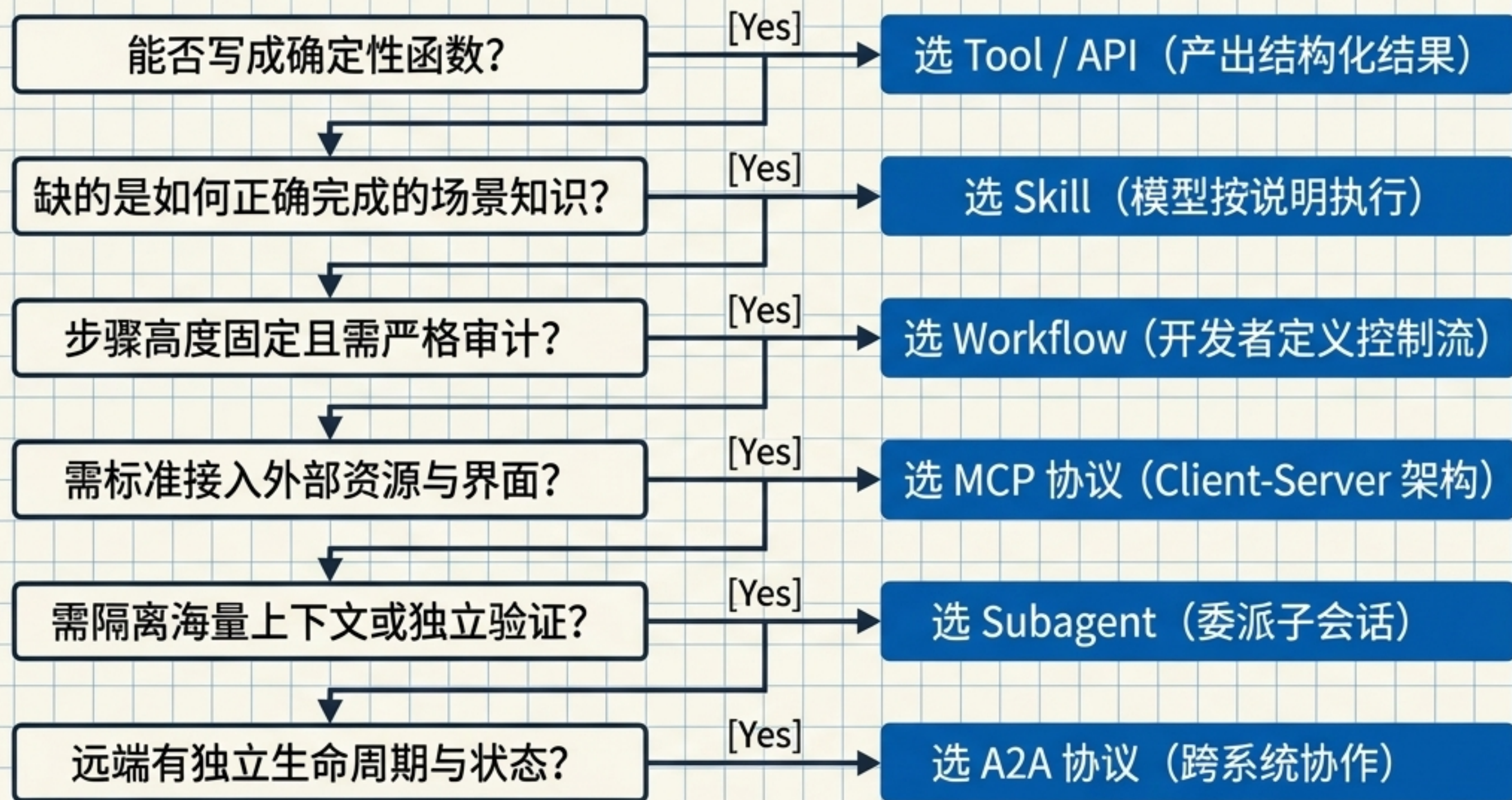
失败恢复位置与幂等键。

最终代码、报告、结构化
数据 (用户的真正目的)。

经确认的偏好与事实。

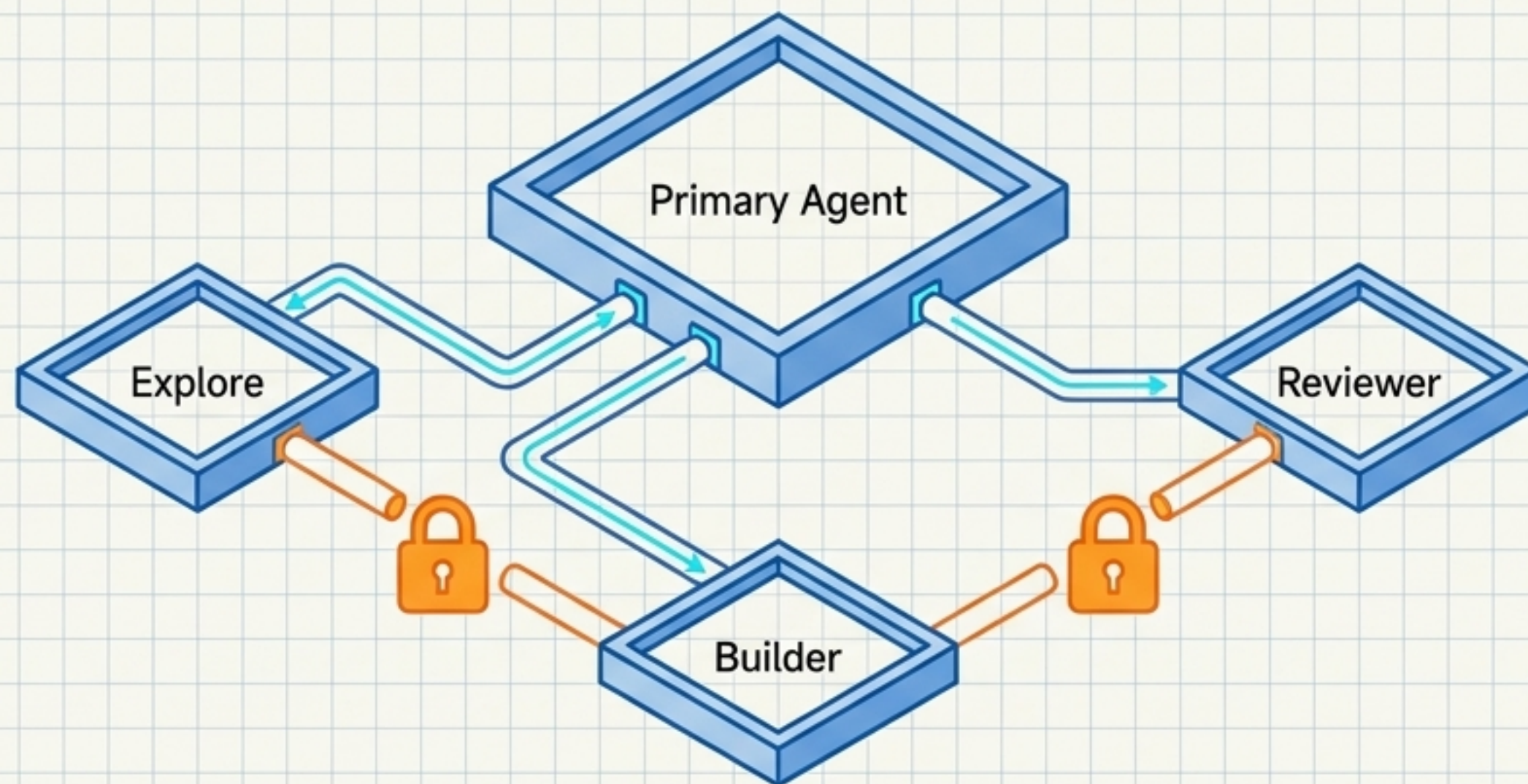
长期记忆准入机制		
必须具：	必须具备：	绝对禁止：
显式确认	✓ 明确来源	✗ 写入临时密钥
	✓ 作用域(个人/团队)	✗ 未经确认的推测
	✓ 版本与纠错机制	✗ 模型产生的“模糊印象”

组件选型矩阵：精准引入多 Agent 与扩展机制



不要因为‘多 Agent’听起来更先进，就把一个可确定的单一函数拆成三个角色互相聊天。

角色隔离：Subagent 的真实存在理由



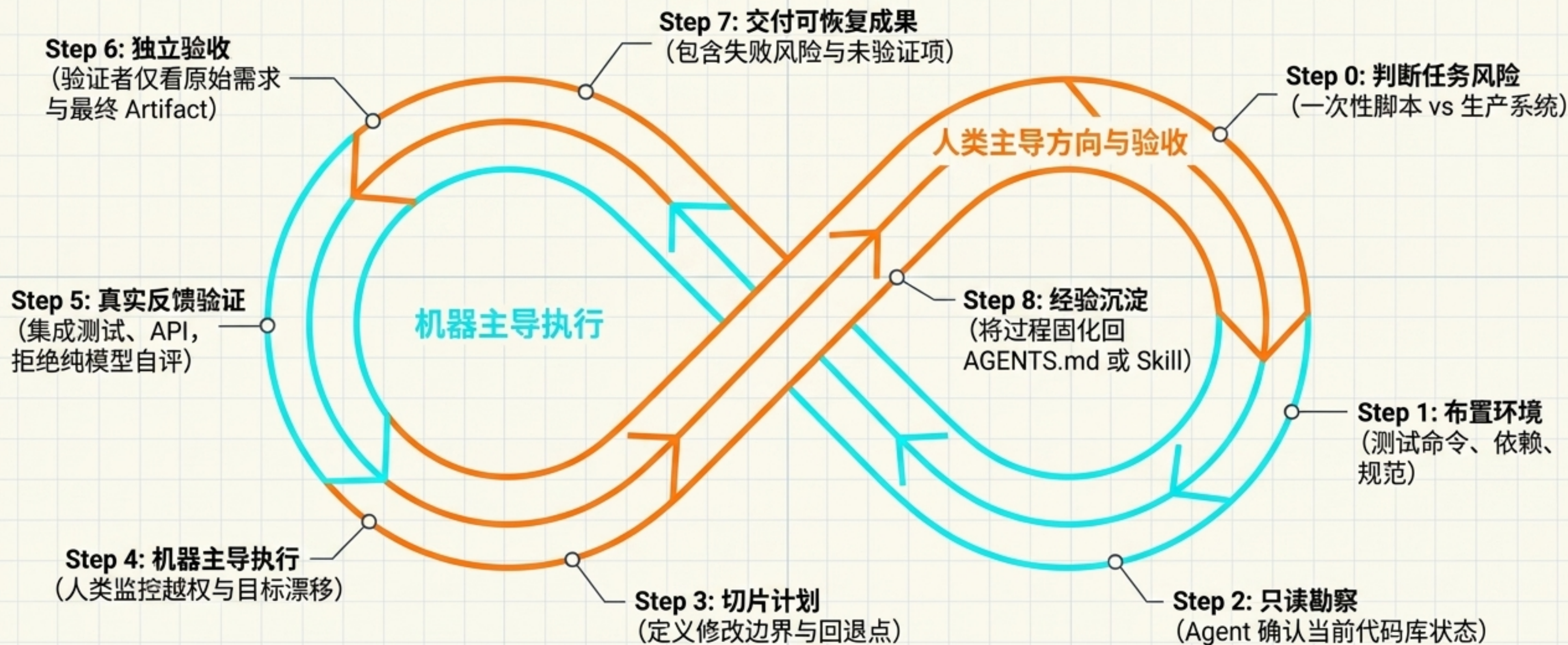
角色定义

- Primary Agent: 管理状态、处理确认、最终验收交付。（不应：无差别复制全局上下文）
- Explore Subagent: 在只读条件下查找代码与方案。（不应：直接修改主工作区）
- Builder Subagent: 实现边界清楚的独立任务。（不应：替 Primary 改变需求）
- Reviewer Subagent: 在独立上下文中检查 Diff 与证据。（不应：继承实现者的自我辩护）

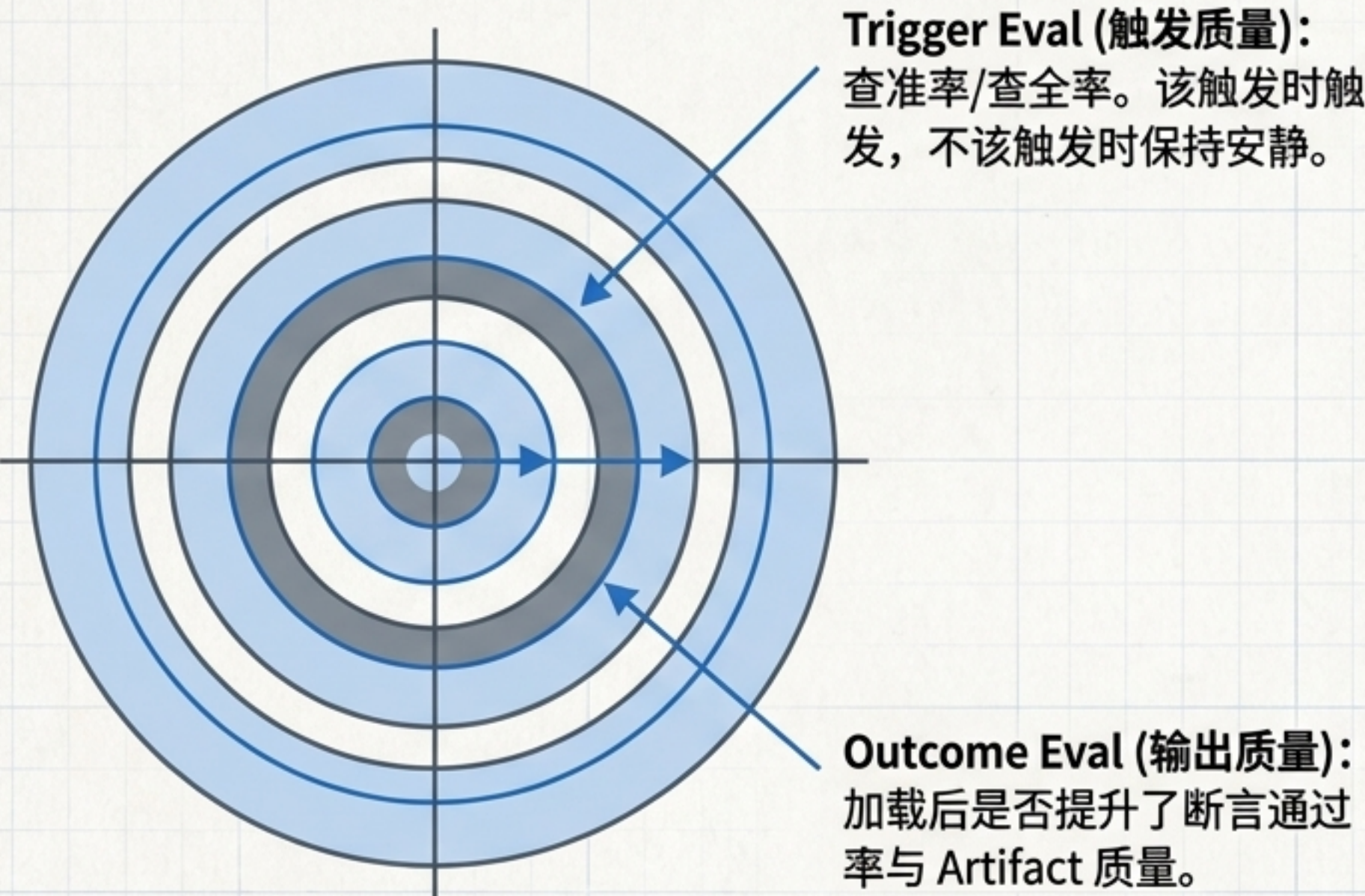
使用 Subagent 的三大充分理由

1. 隔离海量上下文以防主会话污染。
2. 子任务具备真实的并行独立性。
3. 需要在不同信息条件下进行独立验证。

生产级 Agentic Coding: 8 步验证闭环



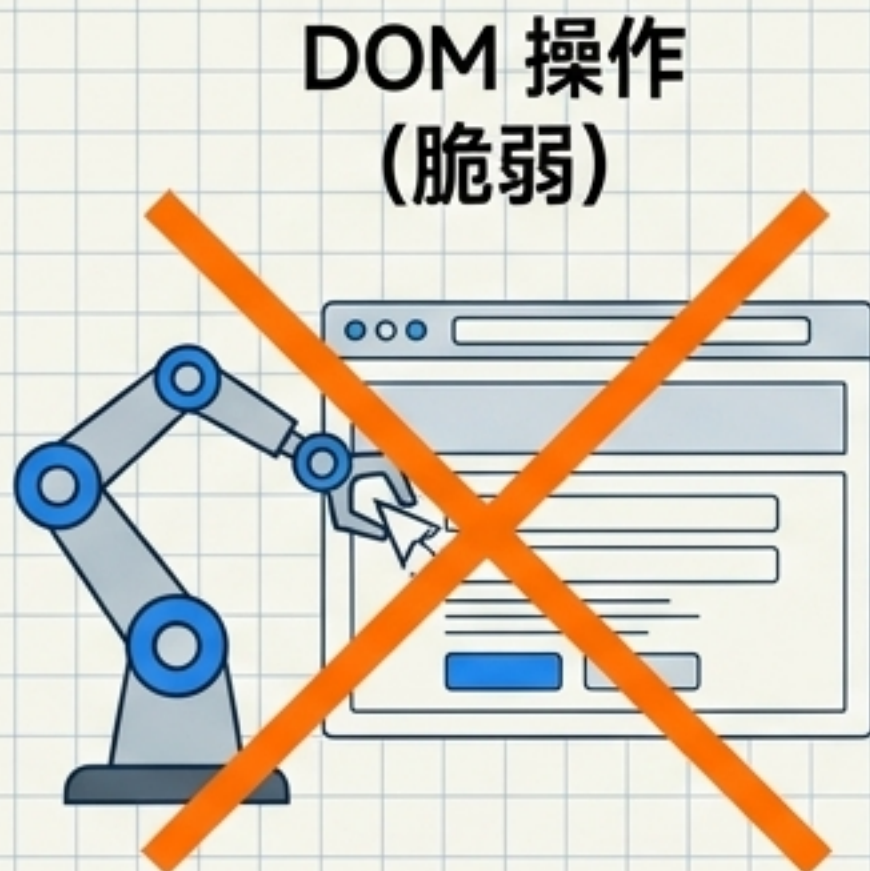
质量防御：Skill 评测矩阵与边界控制



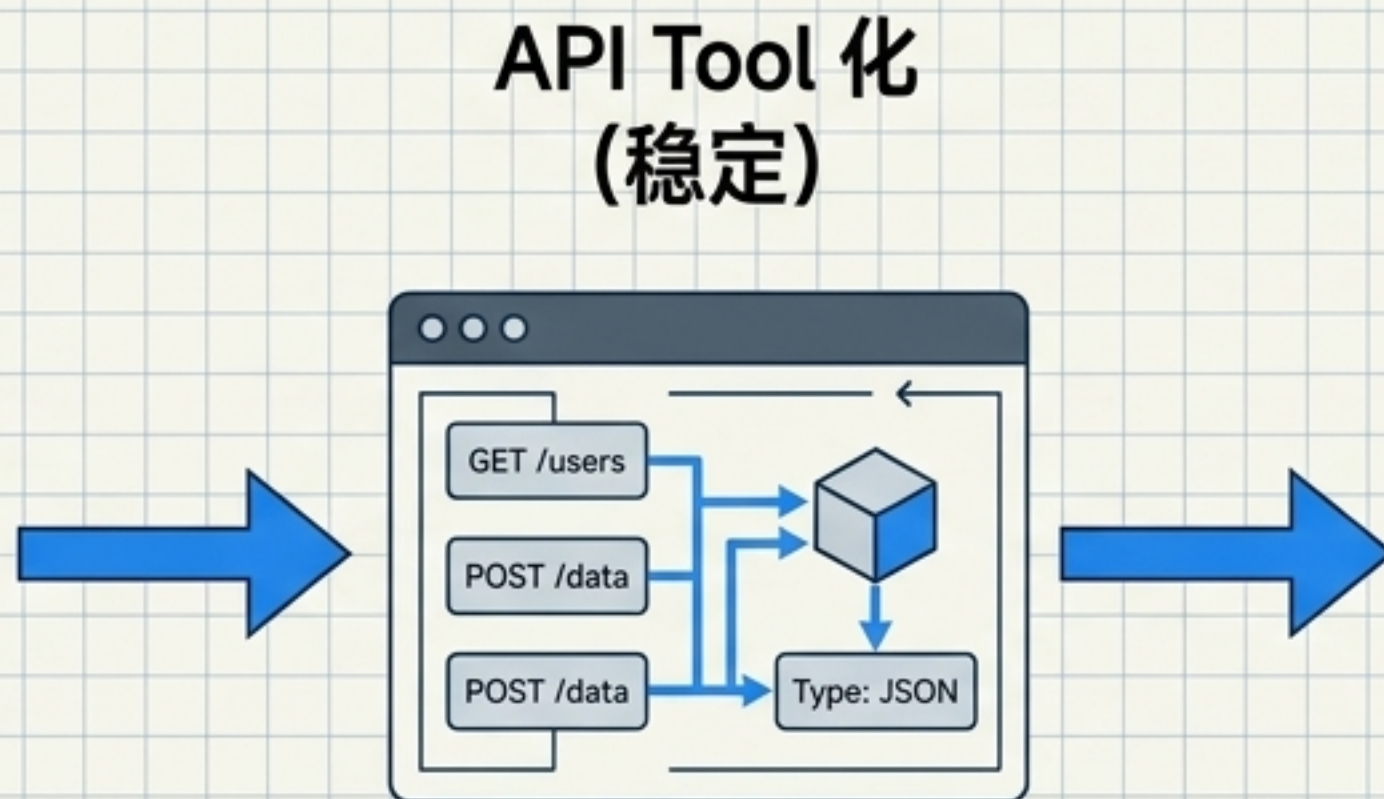
6 类触发测试用例设计		
类别	测试语句	预期
明确点名	使用 \$review 工具检查	必须触发
明确业务意图	帮我审查越权漏洞	自动 Skill 触发
隐含意图	修改配置有什么风险?	视边界而定
关键词近邻 (负样本)	修改页面颜色	不触发
同一文件 不同意图	给文件补注释	不触发
多意图冲突	先修漏洞再发版	触发主 Skill, 分步执行

⚠️ 硬边界保证: ‘Explicit-only’ (仅限明确点名) 必须由底层 Harness 拦截实现, 不能仅靠 Prompt 软约束。

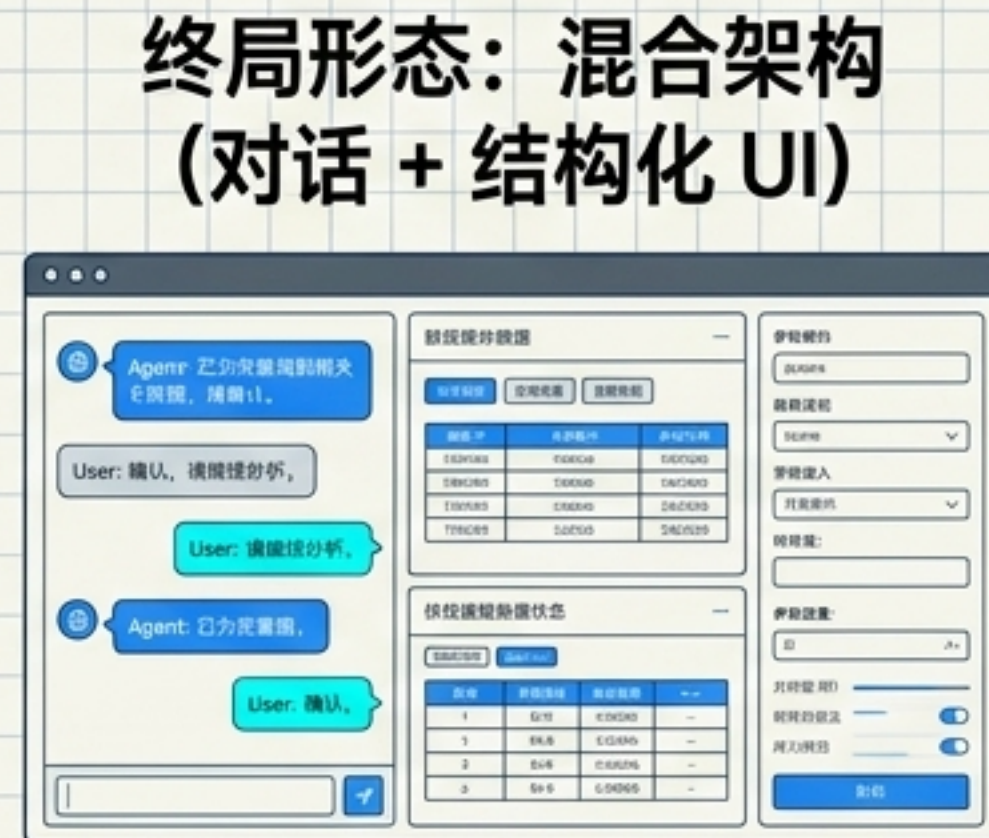
形态跃迁：传统 WebUI 的 Agent 化改造路径



Agent 模拟点击。
慢、难审计、UI 极易失效。



将后端接口封装为 Typed Tools。
快、可测试、便于授权。



Agent 编排意图，卡片/工作区承载精确交互。

破除行业迷思：目标绝不是删掉所有前端按钮只留一个聊天输入框。应复用沉淀的参数校验、状态机与可视化组件。

交互映射：旧 UI 元素向 Agent 的无缝对接

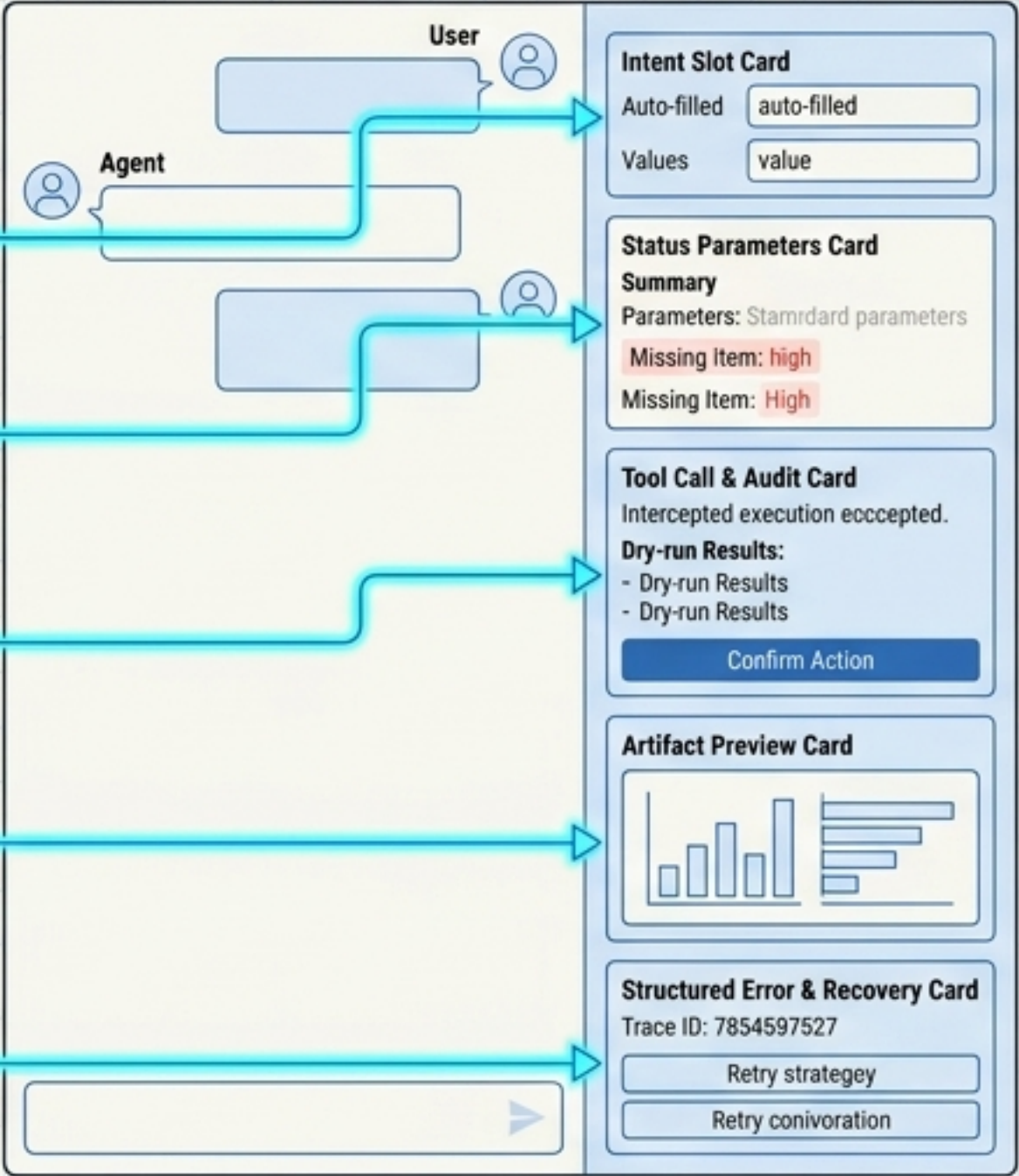
传统 WebUI



传统 WebUI

旧 UI 元素 -> 新 Agent 交互	
文本/下拉框	-> 意图 Slot: 自动从对话推断填充, 高风险值留作显式确认。
多步 Wizard 表单	-> 状态参数卡: 一次性展示解析结果, 仅追问缺失的决定性条件。
运行/提交按钮	-> Tool 调用 + 审计: 拦截执行, 展示参数副作用与 Dry-run 结果。
复杂表格/图表	-> Artifact 预览: 不强迫模型将几百行数据复述成文本, 直接渲染原生图表。
Toast 报错	-> 结构化错误 + 恢复建议: 保留 Trace ID 并由 Agent 提供重试策略。

Agent 混合 UI

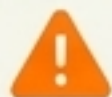


Agent 混合 UI

走向生产环境的 10 条工程公理

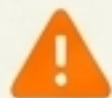
1

Chatbot 暴露说话能力，Agent 暴露在受控环境中持续工作的能力。



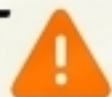
2

RAG 解决“知道什么”，Tool 解决“能做什么”，Skill 解决“应该怎么做”。



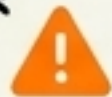
3

把不确定性留给模型，把确定性沉淀到 Harness 与环境。



4

记忆不等于聊天记录，长期记忆必须支持确认、过期与纠错。



5

API 必须先转化为强 Schema 的 Tool，再交由 Agent 调用。

6

stdout 返回纯机器数据，stderr 留给诊断，大结果必须存为 Artifact。



7

Multi-Agent 的本质是为了上下文隔离与独立验证，而非模拟人类组织。

8

高风险边界必须由底层沙箱与权限引擎（Policy）强制接管。



系统设计的最终目标，不是让模型拥有无限自由；而是提供一个反馈真实、状态可恢复、边界不可突破、结果可验证的数字工作世界。